

Taller de Proyecto II
 =====
 Catálogo de Proyectos 2026

G1. ClickAula: Sistema de votación en el aula sin conexión

Introducción

Muchas escuelas no tienen acceso a internet, o prohíben de forma explícita el uso del teléfono celular en el aula. En ese contexto, las asociaciones y equipos de este proyecto construyen un sistema de votación tipo Kahoot que funciona de forma completamente offline. Cada participante dispone de un pulsador con cuatro botones.

Palabras Clave: MQTT; pub/sub; Mosquitto; ESP8266; NodeMCU; Raspberry Pi; access point; hostapd; dnsmasq; sistemas distribuidos; votación; dashboard; IoT;

Materiales

Raspberry Pi 3

NodeMCU (ESP8266) con botones de colores y LED

G2. ParkFinder: Sistema de localización de vehículos en estacionamiento

Introducción

Encontrar el propio vehículo en un estacionamiento de gran capacidad es una experiencia frustrante y frecuente. Los sistemas actuales de guía suelen limitarse a guías estáticas. Este proyecto resuelve ese problema de forma automatizada. Al ingresar, una cámara lee la patente del vehículo mediante OCR y la asocia al lugar físico del vehículo.

Palabras Clave: OCR; visión artificial; HC-SR04; sensores ultrasónicos; Raspberry Pi; estacionamiento inteligente; LED; maqueta; IoT; base de datos.

Materiales

Raspberry Pi 3

Cámara USB o módulo de cámara Raspberry Pi

2 Sensores ultrasónicos HC-SR04 (uno por cochera)

LEDs RGB o bicolor (uno por cochera)

G3. VoiceStock: Relevamiento de inventario por voz con clasificación inteligente

Introducción

Muchos comercios con años de trayectoria acumulan stock sin un registro sistemático: productos mezclados, duplicados con nombres distintos, categorías incorrectas, etc. Este proyecto propone una solución donde el único gesto requerido es hablar. El operario recorre el depósito dictando en lenguaje coloquial lo que encuentra.

Palabras Clave: LLM; tool use; Whisper; STT; TTS; inventario; clasificación semántica; agentes conversacionales; Raspberry Pi; edge AI; Flask; IoT.

Materiales

Raspberry Pi 3

Microfófono USB

Parlante o buzzer con amplificador

G4. SensorHub: Exposición de sensores físicos mediante servidor MCP

Introducción

El protocolo MCP (Model Context Protocol) define un estándar abierto para que los modelos de lenguaje interactúen con sistemas externos de forma estructurada. Un ESP8266 con sensores de temperatura, humedad y movimiento publica sus lecturas a un broker MQTT. Un servidor MCP implementado con FastMCP suscribe a esos datos.

Palabras Clave: MCP; Model Context Protocol; FastMCP; MQTT; ESP8266; sensores; IoT; LLM; tool use; pub/sub; interoperabilidad.

Materiales

ESP8266 (NodeMCU o similar)

Sensor DHT22

Sensor PIR

G5. EcoSort: Clasificador inteligente de residuos en el edge

Introducción

La clasificación incorrecta en puntos de reciclaje es una de las principales causas de contaminación: plásticos en contenedores de vidrio, orgánicos mezclados con otros, etc. Este proyecto propone un clasificador de residuos basado en visión artificial que corre íntegramente en una Raspberry Pi, sin depender de conectividad a internet.

Palabras Clave: visión artificial; edge computing; YOLO; TFLite; clasificación de residuos; Raspberry Pi; servo motor; Flask; IoT; modelos embebidos.

Materiales

Raspberry Pi 4 (recomendada) o 3B+

Cámara USB o módulo de cámara Raspberry Pi

Servo motor (tipo SG90 o similar)

LEDs indicadores (opcionales)

Estructura física de punto de reciclaje

Router o Access Point para red local

Alan https://docs.google.com/document/d/1d89sXkakyx8L_26a7b8-IudhfH7VwxPnLZrce9Q4o5I/edit?tab=t.jfpf1zjcdzxk#heading=h.nsjksy148jdg

A1. Sistema de Acceso Vehicular Seguro: Rolling Code con AES y Transceptor CC1101

Link: <https://www.baeldung.com/cs/rolling-code-security>

Link: <https://www.baeldung.com/cs/replay-attacks>

Introducción

El proyecto consiste en el desarrollo de un sistema de apertura y gestión vehicular que implementa un protocolo de Rolling Code (Código Variable) basado en transceptores de radiofrecuencia.

Materiales

* 2 CC1101

* 2 ESP32

* 1 servomotor sg90 (para simular la apertura de una cerradura)

A2. SmartStep: Clasificación de patologías de pisada mediante TinyML y Series Temporales

Link: <https://www.mercadolibre.com.ar/celda-de-carga-20kg-con-amplificador-hx711-sensor-de-peso/p/MLA46732059>

Link: https://www.google.com/search?q=https://articulo.mercadolibre.com.ar/MLA-2314643616-sensor-de-presion-fsr402-fuerza-sensora-delgada-ar-_JM

Introducción

El objetivo es construir un sistema capaz de diagnosticar si un usuario posee una pisada normal o alguna anomalía (como pie plano o cavo) utilizando técnicas de aprendizaje automático.

Materiales

* 1 Raspberry Pi

* 1 [https://listado.mercadolibre.com.ar/ads1115?sb=all_mercadolibre#D\[A:ADS1115\]](https://listado.mercadolibre.com.ar/ads1115?sb=all_mercadolibre#D[A:ADS1115])

* 2x Sensor de presión FSR 402

* 1 Celdas de carga 20kg + HX711

A3. Impresora CNC con componentes impresos en 3D

Link: <https://www.instructables.com/Arduino-Mini-CNC-Plotter/>

Introducción

El proyecto aborda el diseño y construcción de un plotter CNC de precisión escala A4, utilizando una estructura mecánica basada en piezas impresas en 3D y motores de servo.

Materiales

- * Plotter para montar la estructura (<https://mega.nz/fm/FR9nVJKK>)
- * 3 motores paso a paso <https://www.amazon.com/ELEG00-28BYJ-48-ULN2003-Stepper-Arduino/dp/B01CP18J4A?&linkCode=sl1&tag=mertarduino-20&linkId=870f79f4b690f1>
- * 1 ESP32/Arduino

A4. Sistema de Monitoreo de Área Corporal (BAN) y Detección de "Hombre Caído"

link: <https://github.com/Kunal70616c/Fall-Detection-and-Alert-System-Using-Body-Area-Network/tree/main>

Introducción

El proyecto plantea un dispositivo portable basado en una red de sensores de área corporal (BAN) enfocado en la seguridad operativa, extrayendo y optimizar

Materiales

- * [https://listado.mercadolibre.com.ar/bmp180#D\[A:Bmp180](https://listado.mercadolibre.com.ar/bmp180#D[A:Bmp180)] sensor de presión atmosférica
- * 1 Sensor MPU6050
- * 1 ESP32

A5. Mano Simulada con DMP

Link: <https://industrialmonitordirect.com/es/blogs/knowledgebase/mpu6050-gyro-drift-fix-dmp-kalman-filter-and-i2cdevlib-setup?srsId=AfmBOop7fqVelp-8bdRys>

Introducción

Desarrollar un dispositivo con un ESP32 y un sensor MPU6050 que lea y envíe datos de orientación por red. El objetivo principal es comparar dos formas de c

Materiales

- * 1 ESP32
- * 1 MPU6050

Julián

J1. Brazo Robótico

El proyecto es una continuación del trabajo de años anteriores, donde se busca el desarrollo de un brazo robótico manipulador controlado por microcontrola

Este evolutivo plantea la integración de cinemática inversa con detección de autocolisiones, con el objetivo de lograr la planificación y orquestación de l

Adicionalmente, se incorpora un sistema de "upload" del modelo del brazo hacia el microcontrolador, abstrayendo la lógica de control de los datos específic

Materiales (los mismos que el año pasado):

- Impresiones 3D del brazo con los 5 SG90
- 1 ESP32
- Fuente de alimentación con tira de pines (debidamente aisladas)

J2. Pressure Sensitive Stylus

El objetivo es diseñar e implementar un stylus (lápiz inteligente) con detección de orientación relativa, desplazamiento en el espacio tridimensional y un

La detección de la orientación se realiza por medio de la integración de un filtro del kalman; mientras que el desplazamiento implica proyectar correctame

Toda la lógica se lleva a cabo dentro del MCU desde el cual se enviará por WiFi toda la información del estado del sistema hacia un servidor local.

Adicionalmente, el sistema debe contar con una interfaz gráfica en una SPA sencilla donde permita al usuario realizar un dibujo por medio de este stylus

Por último, se deberá realizar un análisis de los límites de la implementación lograda.

Materiales:

- 1 MCU
- 1 MPU6050
- 1 Sensor Efecto hall (KY-024)

J3. Detección de anomalías en mantenimiento predictivo

El proyecto consiste en armar un sistema de Mantenimiento Predictivo IoT en miniatura simulando un entorno industrial.

Constará de un motor que opera de forma normal sobre el cual se agrega un medidor inercial (IMU).

Se pretende entrenar un pequeño modelo de ML que clasifique la operación del motor como Normal o Anómala. Dicho modelo se ejecuta sobre un MCU.

Adicionalmente, el sistema debe comunicar sus mediciones a un servidor web local que deberá mostrar gráficamente en una línea de tiempo cómo se ve la opera

Materiales:

- 1 MCU
- 1 MPU6050
- 1 Motor DC
- Fuente de alimentación

J4. Infrarrojo (continuación)

El objetivo de este proyecto es ampliar las capacidades de la red construida en el proyecto anterior.

Empleando tres dispositivos con comunicación infrarroja y que sean visibles entre sí, se pretende lograr una red de comunicación entre ellos que admita múl

Para lo cual, se debe lograr una comunicación resiste a ruidos, a colisiones y debe ser capaz de soportar la baja y alta de dispositivos.

Adicionalmente, se debe contar con un servidor central local que reciba y muestre la información de estado de cada dispositivo.

Al final del proyecto, se pretende un análisis de las limitaciones técnicas del sistema.

Materiales (mismo que el año pasado):

- 3 MCUs
- 3 pares receptor/transmisor Infrarrojo

J5. LoRa OTA Update

Se dispone de dos MCU con módulo LoRa para comunicación inalámbrica.

El proyecto pretende explorar la actualización de código ejecutable distribuido por red inalámbrica entre pares (lo que se conoce como OTA)

Cada dispositivo deberá contar con una micro VM que ejecute bytecodes para procesamiento numérico con fines experimentales.

La especificación de este sistema debe ser diseñada por el grupo y debe ser útil para demostrar el funcionamiento correcto del mismo.

Uno de los dos MCUs estará conectado en serie a la PC desde la cual recibirá la nueva versión del código a ejecutar.

Mediante LoRa el MCU retransmitirá el código a su par y se coordinarán para ejecutar la nueva versión.

Materiales:

- 2 MCUs
- 2 módulos LoRa
- Al menos una fuente de alimentación

Merlina

M1: Detección de Presencia

Investigación y análisis experimental centrado en la adquisición, filtrado y evaluación de señales CSI (Channel State Information) del ESP32. El desafío te

- Caso 1: sensor PIR conectado por GPIO al ESP32. Sus métricas (tasa de detección, falsos positivos, latencia) fijan el estándar de referencia. Es el piso

- Caso 2 (CSI con router): un nodo emisor y un nodo receptor ESP32 analizan el canal Wi-Fi a través del router existente, contrastando métricas contra el C

- Caso 3 (CSI sin router): el nodo emisor genera su propia red en modo AP, eliminando el router, para cuantificar el impacto en precisión y calidad de señ

referencia: <https://github.com/francescopace/espectre>

Materiales:

- 5x ESP32
- 1x Sensor PIR
- 1x Router Wi-Fi
- Cables, protoboard

 M2: Tablero Vehicular
 El grupo deberá implementar una pantalla TFT gobernada por un ESP32 que muestre parámetros de un vehículo eléctrico (velocidad, batería, sensores) mientras el piso de aprobación es el sistema funcionando integrado en sus tres capas: pantalla, MQTT e interfaz web. Como diferencial, se puede caracterizar el punto de referencia pantalla: https://github.com/Bodmer/TFT_eSPI

Materiales:
 1x ESP32
 1x Pantalla TFT (táctil)
 1x Motor DC
 1x Driver de motor
 1x Sensor de efecto Hall
 1x Sensor de consumo
 1x Potenciómetro
 1x Fuente de alimentación

 M3: Control Hidráulico
 Prototipo a escala de un lazo de control industrial. El desafío es lograr que el sistema controle una bomba y mida el caudal de forma continua y confiable, el trabajo se estructura en dos fases. La primera es implementar el control continuo sin comunicaciones: bomba y caudalímetro funcionando juntos de forma independiente. Como diferencial, se puede sumar el acelerómetro, explorar el Watchdog Timer, documentar el comportamiento del sistema ante condiciones de red adversas.

Materiales:
 1x ESP32
 1x Bomba de agua
 1x Módulo Mosfet
 1x Fuente de alimentación
 1x Acelerómetro
 1x Caudalímetro
 1x Potenciómetro

Fernando

 F1 - Sensado de Consumo - Multímetro "Automático"
 Idem año pasado

F2 - Receptores de Radio FM HEX3653 y TEA5767: extensión del proyecto
 Unificación de salida de audio con amplificador digital, multiplexación/aislación de salidas analógicas.
 Definición de interfaz para cada placa o unificación de interfaz para ambas placas.
 Evaluación completa de la interfaz con eventos asíncronos usando AJAX
 Opcional: interfaz responsiva/adaptativa para móviles y computadoras

F3 - Evaluación de sensores de distancia de ultrasonido para evitar colisiones
 Cantidades, tamaños de objetos identificados, velocidades posibles
 En principio montaje autónomo de entre 1 y 4 sensores HC-SR04 (el más común)
 Quizás con un autito para demostrar funcionamiento final
 Opcional: montaje en un servo para "escena"

F4 - Problema del WDT - Origen y Soluciones Posibles
 >>que se conoce del problema de reset por wdt en el esp32?
 El "reset por WDT" (Watchdog Timer) en el ESP32 es un problema bastante común, pero no es un bug en sí: es un mecanismo de protección que indica que algo está mal.
 🚫 Qué es el WDT en ESP32
 El watchdog es un temporizador de hardware que espera que tu código "respire" periódicamente. Si no lo hacés (porque el sistema se cuelga o bloquea), fuerza un reset automático.
 En ESP32 hay principalmente dos tipos:
 Interrupt WDT → detecta cuando el sistema deja de atender interrupciones
 Task WDT (TWDT) → detecta tareas de FreeRTOS que no "ceden" CPU
 ⚠️ Por qué ocurre el reset (causas típicas)
 1. Bloqueos o loops infinitos
 La causa más frecuente:
 while(true) sin delay() o yield()
 tareas que nunca liberan CPU
 🛠️ Resultado: el scheduler no corre → watchdog dispara reset.
 💡 Ejemplo típico:
 Agregar vTaskDelay() o delay() suele arreglarlo.
 2. Tareas que tardan demasiado
 Si una función tarda más que el timeout del WDT:
 operaciones de red (WiFi, HTTP)
 acceso a sensores lento
 cálculos pesados
 🛠️ el watchdog interpreta que el sistema se colgó.
 Ejemplo real: un WiFi.scanNetworks() puede disparar WDT si bloquea demasiado
 3. Problemas con multitarea (FreeRTOS)
 El ESP32 es dual-core y usa RTOS:
 tareas mal priorizadas
 starvation (una tarea monopoliza CPU)
 tareas en el core equivocado
 🛠️ alguna tarea crítica no corre → watchdog salta
 4. No "alimentar" el watchdog
 Si activás el WDT manualmente:
 tenés que hacer esp_task_wdt_reset() o equivalente
 si no lo hacés → reset garantizado
 5. Uso incorrecto de hardware (pines / flash)
 Casos menos obvios:
 usar pines reservados para flash
 bloquear acceso a memoria
 🛠️ el CPU no puede ejecutar código → WDT reset
 Ejemplo: usar GPIO de SPI flash puede causar TG1WDT_SYS_RESET
 6. Interrupciones o código crítico mal diseñado
 ISR muy largas
 deshabilitar interrupciones demasiado tiempo
 🛠️ dispara el Interrupt WDT
 7. Problemas más sutiles
 deadlocks entre tareas

cache/flash bloqueados (casos avanzados)
 librerías mal implementadas
 deep sleep / wakeup inconsistentes
 🔍 Mensajes típicos que vas a ver
 Task watchdog got triggered
 TG1WDT_SYS_RESET
 Interrupt wdt timeout on CPU0
 Cada uno apunta a un tipo distinto de watchdog.
 🛠️ Cómo se suele solucionar
 ✅ Buenas prácticas básicas
 Nunca loops infinitos sin delay() o yield()
 Dividir tareas largas
 Usar vTaskDelay() en FreeRTOS
 ✅ Para tareas pesadas
 usar tareas separadas
 evitar bloquear el loop principal
 usar timeouts en WiFi / I/O
 ✅ Debug
 usar esp_reset_reason()
 analizar backtrace (ESP Exception Decoder)
 ver qué tarea causó el WDT
 ✅ Ajustes avanzados
 aumentar timeout del watchdog
 deshabilitarlo (solo para testing ⚠️)
 reestructurar prioridades de tareas
 🌟 Idea clave (lo más importante)
 Un reset por WDT no es el problema real, es el síntoma.
 📌 Siempre significa alguna de estas:
 tu código se quedó bloqueado
 una tarea monopolizó CPU
 algo impidió que el sistema “respire”
 Si querés, podés pegar tu log de error (rst:0x... o task_wdt) o tu código y te ayudo a encontrar la causa exacta, que suele ser bastante específica en cada caso.
 >>se conoce si aparece mucho cuando se usa dht11?
 Si hay bastantes reportes de resets por WDT cuando se usa el sensor DHT11, pero no porque el sensor “rompa” el ESP32, sino por cómo se lo lee.
 🟡 Por qué pasa con el DHT11
 El DHT11 usa un protocolo muy sensible al timing (bit-banging), que suele implicar:
 deshabilitar interrupciones por unos microsegundos
 loops de espera muy ajustados (busy wait)
 En el ESP32 eso choca con:
 FreeRTOS (multitarea)
 WiFi/Bluetooth (muy dependientes de timing)
 watchdog activo
 📌 Resultado: si la lectura bloquea demasiado tiempo o se repite muy seguido, el sistema no atiende al scheduler → WDT reset.
 ⚠️ Situaciones donde aparece más
 1. Lecturas muy frecuentes
 El DHT11 no está diseñado para lecturas rápidas:
 mínimo recomendado: ~1 segundo (mejor 2 s)
 Si hacés lecturas tipo:
 loop() {
 dht.readTemperature();
 }
 📌 altísima probabilidad de WDT
 2. Librerías no optimizadas para ESP32
 Muchas librerías populares (como las pensadas para Arduino UNO) hacen:
 busy-wait largo
 bloqueos sin yield()
 📌 en ESP32 eso puede disparar:
 Task watchdog got triggered
 3. Uso junto con WiFi
 Muy típico:
 ESP32 conectado a WiFi
 lectura de DHT11 en loop
 📌 el WiFi necesita CPU constantemente
 📌 si lo bloqueás → watchdog
 4. Código dentro de tareas sin delay
 Ejemplo problemático:
 while (true) {
 dht.readHumidity();
 }
 📌 no hay vTaskDelay() → WDT casi seguro
 🛠️ Cómo evitarlo
 ✅ 1. Reducir frecuencia de lectura
 delay(2000);
 o con millis()
 ✅ 2. Usar librería compatible con ESP32
 Algunas versiones de DHT incluyen yield() internamente o están adaptadas.
 ✅ 3. No bloquear el loop
 Si usás FreeRTOS:
 vTaskDelay(pdMS_TO_TICKS(2000));
 ✅ 4. Separar en tarea de baja prioridad
 Leer el DHT en una task dedicada ayuda a no bloquear otras.
 ✅ 5. Alternativa (muy recomendada)
 El DHT11 es bastante limitado. Muchos problemas desaparecen usando:
 DHT22 (mejor timing)
 sensores I2C como BME280 (mucho más estables)
 🌟 Resumen claro
 Si, el DHT11 está asociado frecuentemente a resets por WDT en ESP32, pero:
 📌 no es un fallo del hardware
 📌 es un problema de timing + bloqueos + frecuencia de lectura

F5 - Reconexión WiFi ESP32

Replicación de problemas y soluciones en cada caso

La desconexión/baja de WiFi es un problema conocido del ESP32. Se debe probar tanto en modo STA como en modo AP. Al menos en modo STA se puede simular apaga

Condiciones generales para todos los proyectos F<n>:

- IDE Arduino. Pueden desarrollar en el ambiente que prefieran, pero los fuentes entregables deben ser proyectos Arduino. En todos los casos se entrega un
- Bitácora desde el 1er día, toda consulta/definición de cátedra documentada (incluyendo fotos de lo que se explique en el pizarrón)
- Todo lo entregado en una carpeta almacenada en la nube (mega, gdrive, dropbox, etc.) con acceso irrestricto para la cátedra.
- Clases y evaluaciones en horarios de clase de los lunes, a menos que se indique otra cosa de manera explícita.